

Update - Differential Analysis of LLMs in Text2SQL Generation

Jaykumar Patel
patel.jay4802@utexas.edu

An-Jung Huang
ajhuang@utexas.edu

Ryan Kolm
ryankolm@utexas.edu

1 Introduction

As large language models (LLMs) continue to evolve and improve, an interesting application of LLMs is in the generation of SQL queries from natural language. This is known as Text2SQL. Text2SQL can enable non-experts to extract valuable information from a database without having to learn complex query languages like SQL. By simply describing the data they want in plain English, users can leverage LLMs to translate their requests into structured queries that interact directly with a database. This has the potential to democratize data access across organizations, reduce reliance on technical teams for routine data retrieval tasks, and improve overall efficiency in data-driven decision-making.

In this project, we compare the performance of different LLMs in generating SQL queries from natural language. We explore 2 different frameworks, with (Agent-based) and without AI agents (Agentless), and 3 different LLMs, one of which is GPT-4o and the others are to-be-decided. In the Agent-based framework, there are 3 agents that handle specific tasks such as determining the number of relevant tables, retrieving the schemas of the relevant tables, and generating a SQL query given a natural language query. In the Agent-less framework, the schemas of all tables in the database (relevant and irrelevant) is passed as content to the LLM when generating the SQL query given the natural language query. We run our Agent-based and Agentless frameworks with 3 different LLMs on 2 benchmarks – Chinook sample database and BIRD-SQL Dev Set.

2 Previous Work

...

3 Methodology

In this section, we outline the methodology used for our differential analysis of LLMs in Text2SQL generation.

3.1 Model Selection

For this study, we selected the three Large Language Models (LLMs) of varying architecture and scale: GPT-4o mini, Llama 3 8B, and TBD.

The default temperature for GPT-4o mini is 0.7; however, in this study, we set the temperature to 1.0. Temperature controls the level of randomness in the text generated by large language models during inference (<https://www.ibm.com/think/topics/llm-temperature>). By increasing the temperature to 1.0, we aim to encourage greater diversity in the SQL queries generated across three iterations, increasing the likelihood that at least one of them yields a correct result.

3.2 Framework Selection

There are many ways that Text2SQL can be implemented. In this study, we compare the performance difference between 2 frameworks, one that utilizes a series of AI agents (Agent-based), and one that performs without AI agents (Agentless).

1. **Agentless:** In this framework, an LLM to generate the response directly. This is synonymous with the use of a chatbot. For this framework, we will provide the schemas of every table in the database and the natural language query to the LLM. Then we will run the generated SQL query and evaluate the results based on accuracy, execution time, and resource utilization.
2. **Agent-based:** In this framework, there are distinct agents that each play a different role in the generation of the response. There are 3 agents: **Relevance Agent**, **Retrieval Agent**, and **Query Agent**. The **Relevance Agent** is responsible for determining how many tables are relevant to a natural language query. The **Schema Agent** will be responsible for retrieving the schemas of the relevant tables from the database given the natural language query. The **Query Agent** will be responsible for generating the SQL query given the relevant schemas and the natural language query. Then we will

run the generated SQL query and evaluate the results based on the same criteria as the Agentless framework. The agent-based pipeline can be seen in 1

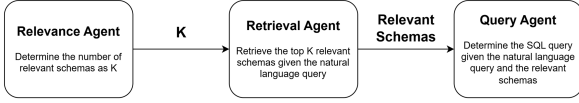


Figure 1: Agent-based Text2SQL Pipeline

The main difference between Agentless and Agent-based framework is that during the Text2SQL generation, the schemas of all tables are used for Agentless, whereas only the schemas of relevant tables are used for Agent-based. The comparison between these frameworks can provide insight into the trade-offs between simplicity and modularity, particularly in terms of scalability and accuracy – considering Agentless make less calls to the LLM than Agent-based. Specifically, we aim to explore whether introducing specialized agents leads to more accurate query generation, or if the added complexity results in diminishing returns compared to a simpler, monolithic approach. This evaluation can help guide future development of LLM-powered database interfaces.

3.3 Benchmark Dataset Selection

The SQL databases that we plan to use are the Chinook sample database, which is a sample database provided by SQLite, and the BIRD-SQL Dev Set. For each of the tables in the Chinook databases, we will create the Schemas, which will be used as the context passed to the LLMs. The BIRD-SQL Dev Set include the Schemas of the tables in its database. Additionally, we will determine 10 natural language queries for Chinook and 15 for BIRD-SQL, their corresponding SQL queries, and the results of executing the SQL queries. Using that benchmark dataset as ground truth, we will perform a differential analysis of the performance of 3 LLMs for the 2 frameworks. We will analyze the ability of the LLMs to generate correct SQL queries, as well as the differences in performance between the LLMs and frameworks.

We will test every Model+Framework combination on every natural language query 3 times. This will allow us to determine the consistency of the Model+Framework in generating SQL queries.

4 Results

For this update, we have run the ‘Agentless’ and ‘Agent-based’ framework with GPT-4o on Chinook and BIRD-SQL. The Chinook benchmark includes 10 natural language

queries and the BIRD-SQL benchmark includes 15 natural language queries (7 simple, 5 moderate, and 3 challenging – categorizations provided by BIRD-SQL).

4.1 Correctness and Consistency

4.1.1 Chinook

4.1.2 BIRD-SQL

ID	Query Type	T _r Agent-based	T _r Agentless
1	Simple	3/3	2/3
2	Simple	3/3	3/3
3	Simple	0/3	0/3
4	Simple	3/3	3/3
5	Simple	3/3	3/3
6	Simple	1/3	0/3
7	Simple	3/3	3/3
8	Moderate	3/3	3/3
9	Moderate	0/3	1/3
10	Moderate	3/3	3/3
11	Moderate	0/3	0/3
12	Moderate	3/3	3/3
13	Challenging	0/3	0/3
14	Challenging	0/3	0/3
15	Challenging	2/3	0/3

Figure 2: GPT-4o mini results on BIRD-SQL

The results from the study on the BIRD-SQL benchmark can be seen in Figure 2. Firstly, the results demonstrate the Text2SQL is a challenging task. For some questions, including a simple one, neither frameworks were able to generate a correct SQL query even once. Secondly, the results reveal that most of the time, the Agent-based system is more consistently correct, shown by the rows with ID 1, 6, and 15. Agentless was able to generate a solution when Agent-based failed only once, shown by the row with ID 9.

Agents are employed to retrieve relevant schemas for the Agent-based system. These relevant schemas are used during the Text2SQL generation. However, for Agentless system, all schemas are used during the Text2SQL generation. This means that irrelevant schemas could be passed as context to the LLM, resulting in incorrect SQL queries. This is consistent with the findings from Figure 2.